

Felipe Silvestre Cardoso Roberto
170425

Implementação de um Processador RISC-V com Utilização do kit FPGA

São José dos Campos - Brasil

Abril de 2025

Felipe Silvestre Cardoso Roberto
170425

Implementação de um Processador RISC-V com Utilização do kit FPGA

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Abril de 2025

Resumo

Este relatório apresenta a arquitetura RISC-V e as definições desenvolvidas pelo autor, destacando sua filosofia minimalista, modularidade aberta e aplicabilidade desde sistemas embarcados até computadores de alto desempenho. Na fundamentação teórica, são discutidos os paradigmas CISC vs RISC, modelos de memória Von Neumann e Harvard, além de conceitos de pipeline e execução de instruções. Em seguida, descreve-se a ISA base RV32I, seus formatos de instrução e as extensões padrão. No desenvolvimento, detalham-se as definições próprias do conjunto de instruções e o projeto do caminho de dados de um processador monociclo implementado em FPGA. Espera-se validar funcionalmente o projeto, analisar desempenho e custo em termos de área de silício e consumo de energia, e evidenciar a extensibilidade e vantagens competitivas do RISC-V.

Palavras-chaves: Arquitetura e Organização de Computadores. RISC-V. Kit FPGA.

Lista de ilustrações

Figura 1 – Arquitetura de Von Neumann e de Harvard	10
Figura 2 – Árvore de decisão para decodificação baseada em <code>opcode</code> e <code>funct</code> . . .	15
Figura 3 – Caminho de dados geral do processador	21
Figura 4 – Caminho de dados para instruções do tipo R	21
Figura 5 – Caminho de dados para instruções do tipo I	22
Figura 6 – Caminho de dados para instruções do tipo J	22
Figura 7 – Caminho de dados para instruções do tipo B	23
Figura 8 – Waveform de Simulação da ULA.	31
Figura 9 – Waveform de Simulação do Banco de Registradores.	32
Figura 10 – Waveform de Simulação da Memória RAM.	32
Figura 11 – Waveform de Simulação da Memória de Programa.	33
Figura 12 – Waveform de Simulação da Integração da ULA com o Banco de Regis- tradores.	33

Lista de tabelas

Tabela 1 – Formato das Instruções do Tipo R	16
Tabela 2 – Instruções do Tipo R implementadas	16
Tabela 3 – Formato das Instruções do Tipo I	16
Tabela 4 – Instruções do Tipo I implementadas	17
Tabela 5 – Formato das Instruções do Tipo J	17
Tabela 6 – Instruções do Tipo J implementadas	17
Tabela 7 – Formato das Instruções do Tipo B	18
Tabela 8 – Instruções do Tipo B implementadas	18
Tabela 9 – Formato das Instruções do Tipo N	18
Tabela 10 – Instruções do Tipo N implementadas	18
Tabela 11 – Formato das Instruções do Tipo IO	19
Tabela 12 – Instruções do Tipo IO implementadas	19
Tabela 13 – Resumo das Instruções Implementadas	20

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	8
2.1	Geral	8
2.2	Específicos	8
3	FUNDAMENTAÇÃO TEÓRICA	9
3.1	Arquiteturas CISC e RISC	9
3.1.1	Arquitetura CISC	9
3.1.2	Arquitetura RISC	9
3.2	Arquitetura de Von Neumann e Harvard	10
3.3	Processador RISC-V	11
3.3.1	Arquitetura Básica	11
3.3.2	Formato de Instruções	11
3.3.3	Modos de Endereçamento	12
3.3.3.1	Registrador	12
3.3.3.2	Imediato	12
3.3.3.3	Base-Deslocamento	12
3.3.3.4	Relativo ao PC	12
3.3.3.5	Direto	13
3.4	Linguagem de Descrição de Hardware	13
3.5	Field Programmable Gate Array (FPGA)	13
4	DESENVOLVIMENTO	14
4.1	Características do Processador	14
4.2	Organização do opcode e funct	14
4.3	Conjunto de Instruções	15
4.3.1	Tipo R	15
4.3.2	Tipo I	16
4.3.2.1	Tipo J	17
4.3.2.2	Tipo B	17
4.3.2.3	Tipo N e Tipo IO	18
4.4	Caminho de Dados	20
4.4.1	Caminho de Dados Geral	21
4.4.2	Caminho de Dados para Instruções do Tipo R	21
4.4.3	Caminho de Dados para Instruções do Tipo I	22

4.4.4	Caminho de Dados para Instruções do Tipo J	22
4.4.5	Caminho de Dados para Instruções do Tipo B	23
4.5	Implementação	23
4.5.1	Unidade Lógica Aritmética (ULA)	24
4.5.2	Banco de Registradores	25
4.5.3	Memória RAM	26
4.5.4	Memória de Programa	27
4.5.5	Módulos de Interface e Controle	27
4.5.5.1	Sistema de Entrada e Saída (I/O)	27
4.5.5.2	Gerenciamento de Clock	28
4.5.5.3	Inicialização do Processador (Reset)	28
4.6	Integração dos Módulos	29
5	RESULTADOS OBTIDOS E DISCUSSÕES	31
5.0.1	Waveform da Unidade Lógica Aritmética (ULA)	31
5.0.2	Waveform do Banco de Registradores	32
5.0.3	Waveform da Memória RAM	32
5.0.4	Waveform da Memória de Programa	33
5.0.5	Waveform da Integração ULA com Banco de Registradores	33
5.0.6	Validação Funcional do Processador Integrado em FPGA	33
6	CONSIDERAÇÕES FINAIS	35
	REFERÊNCIAS	36

1 Introdução

A arquitetura RISC-V surgiu na Universidade da Califórnia, Berkeley, em 2010 como uma ISA (Instruction Set Architecture) aberta e livre de royalties. Seu design minimalista, baseado em princípios RISC clássicos, oferece apenas um núcleo compacto com sete instruções básicas, ao qual se acoplam extensões modulares conforme a necessidade de cada aplicação. Essa filosofia possibilita reduzida complexidade de decodificação, menor área de silício, consumo de energia otimizado e maior previsibilidade de desempenho, características essenciais para soluções que vão de sistemas embarcados de baixo consumo a clusters de alto desempenho em supercomputadores.

A motivação central para a escolha do RISC-V neste trabalho reside em suas vantagens competitivas: custeio reduzido por ser aberto, flexibilidade para incluir apenas as extensões exigidas, além de um ecossistema crescente de ferramentas e comunidades acadêmicas e industriais. Essas qualidades permitem ao projetista balancear desempenho, área e consumo de energia de forma personalizada.

Este relatório incorpora ainda as definições desenvolvidas pelo autor para o conjunto de instruções e o caminho de dados do processador. Entre elas, destacam-se:

Definição de formatos de instrução específicos para reduzir o número de bits redundantes e otimizar a decodificação;

Inclusão de operações de manipulação de bits e registro imediato conforme as necessidades do projeto;

Arquitetura de caminho de dados monociclo em FPGA, projetada para suportar as instruções customizadas com mínimo impacto em área e tempos críticos de sinal.

Na fundamentação teórica, são abordados os paradigmas CISC vs RISC, modelos de memória, pipeline e técnicas de execução de instruções. Em seguida, detalha-se a ISA base RV32I, as extensões-padrão e as customizações propostas. No desenvolvimento, o foco recai sobre o formato das instruções e o projeto do caminho de dados monociclo, enquanto os resultados apresentam a validação funcional, medições de desempenho e análise de custos de implementação. Este relatório explora a fundo a ISA base RV32I, suas extensões-padrão e as definições personalizadas elaboradas pelo autor. Na fundamentação teórica, são apresentados os conceitos de CISC vs RISC, modelos de memória, pipeline e execução de instruções. No desenvolvimento, detalham-se o conjunto de instruções definido, o formato das instruções e o caminho de dados de um processador monociclo implementado em FPGA. Por fim, serão avaliados os resultados de funcionalidade, desempenho e custo de implementação, evidenciando as vantagens de adoção do RISC-V.

2 Objetivos

2.1 Geral

O objetivo geral deste trabalho é desenvolver, a partir do conhecimento teórico adquirido na disciplina de Arquitetura e Organização de Computadores, um processador monociclo RISC-V implementado em FPGA, capaz de executar corretamente instruções dos formatos R, I, J e B.

2.2 Específicos

Nesta seção, serão numerados cada um dos objetivos propostos para este projeto:

1. Estudar e comparar diferentes arquiteturas e *designs* de processadores RISC e CISC, de modo a selecionar o modelo mais adequado como base para a implementação.
2. Definir o conjunto de instruções a ser suportado (aritméticas, load/store, desvios e saltos) e especificar os formatos de cada tipo (R, I, J e B) para a decodificação pelo processador.
3. Projetar o *datapath* monociclo completo, contemplando unidades de registradores, ALU, memória de instruções e dados, e barramentos de controle.
4. Desenvolver a unidade de controle capaz de gerar os sinais necessários para cada instrução, incluindo sinais de desvio, acesso à memória e escrita em registradores.
5. Implementar todo o projeto em HDL (Verilog), sintetizar no FPGA e validar funcionalmente o processador por meio de *benchmarks* de teste.

3 Fundamentação Teórica

3.1 Arquiteturas CISC e RISC

De forma geral, um processador é um circuito integrado capaz de executar operações lógicas e aritméticas, além de controlar o fluxo de dados entre memória e periféricos (STALLINGS, 2017). As arquiteturas de conjuntos de instruções definem como o hardware interpreta e executa cada comando de máquina, impactando diretamente a complexidade do projeto e o desempenho do processador.

3.1.1 Arquitetura CISC

A filosofia CISC (Complex Instruction Set Computer) concentra um grande número de instruções, muitas delas de complexidade elevada, que podem executar múltiplas operações em uma única chamada. Características principais:

- **Instruções complexas:** permitem tarefas como manipulação de strings ou acesso indireto à memória em uma só instrução.
- **Microcódigo:** implementação interna é geralmente baseada em microinstruções armazenadas em ROM.
- **Tamanho variável de instrução:** formatos de 1 a vários bytes, dificultando o pipeline.
- **Vantagem:** código de máquina mais compacto, reduzindo o espaço de memória.
- **Desvantagem:** ciclagem mais longa por instrução e pipeline mais complexo (STALLINGS, 2017).

3.1.2 Arquitetura RISC

RISC (Reduced Instruction Set Computer) adota um conjunto reduzido de instruções simples e de formato fixo, favorecendo execuções rápidas e pipelines profundos. Principais aspectos:

- **Formato fixo:** todas as instruções têm o mesmo tamanho (geralmente 32 bits), simplificando a decodificação.
- **Load/store:** apenas instruções de *load* e *store* acessam a memória; operações aritméticas só operam em registradores.

- **Pipeline eficiente:** instruções simples permitem estágios bem definidos e alta frequência de clock.
- **Compilador:** complexidade de tradução fica a cargo do compilador, que gera sequências de instruções simples.
- **Exemplo:** o subconjunto RV32I do RISC-V é composto por cerca de 47 instruções básicas, facilitando o design do processador e garantindo extensibilidade (??).

3.2 Arquitetura de Von Neumann e Harvard

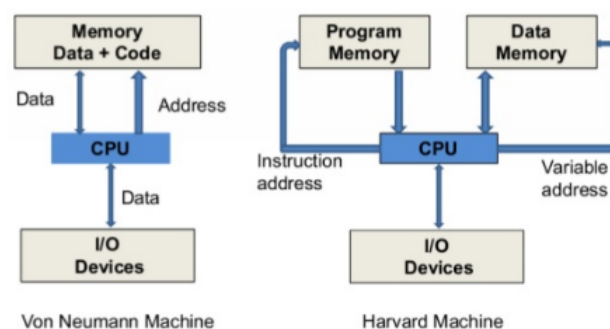
Existem dois modelos clássicos de organização de memória em computadores:

Von Neumann utiliza um único barramento e uma memória unificada para instruções e dados. Cada ciclo de acesso à memória atende tanto à leitura de instruções quanto à leitura/gravação de dados. Essa uniformidade reduz o custo de hardware, mas pode gerar o “gargalo de von Neumann” quando instruções e dados competem pelo mesmo barramento (STALLINGS, 2017).

Harvard separa fisicamente as memórias e os barramentos de instruções e dados. Isso permite acessos simultâneos, aumentando a taxa de transferência e reduzindo contenção. No entanto, há maior custo de implementação e complexidade de design (??).

Para processadores RISC-V monociclo em FPGA, o modelo Harvard é frequentemente adotado, pois facilita a separação clara entre fetching de instruções e operações de dados, melhorando o desempenho mesmo em implementações simples.

Figura 1 – Arquitetura de Von Neumann e de Harvard



Fonte: (EDITORIAIS DA TMLT, 2021)

3.3 Processador RISC-V

O RISC-V é uma arquitetura de conjunto de instruções aberta (ISA) baseada em princípios RISC, desenvolvida na Universidade da Califórnia, Berkeley, em 2010. Seu design modular permite vários níveis de extensão, sendo o núcleo RV32I o alicerce mínimo, composto por instruções simples de 32 bits, que facilitam a implementação em hardware e o pipeline eficiente (??).

3.3.1 Arquitetura Básica

A arquitetura RISC-V segue o modelo Harvard modificado, com memória de instruções e dados separadas, e barramentos independentes para fetching e acesso a dados. O conjunto mínimo RV32I possui:

- 32 registradores gerais de 32 bits (`x0` a `x31`), sendo `x0` sempre zero.
- Unidade Lógica-Aritmética (ALU) capaz de operações inteiras básicas.
- Barramento de instruções de 32 bits, com decodificação fixa.
- Extensões opcionais (M, A, F, D, etc.) que adicionam instruções de multiplicação, atomics e ponto flutuante.

3.3.2 Formato de Instruções

A arquitetura RISC-V ([PATTERSON; WATERMAN, 2019](#)) segue um modelo de instruções de tamanho fixo de 32 bits para seu conjunto base RV32I. Cada instrução é codificada em um dos seis formatos principais: R, I, S, B, U e J. Cada formato organiza os 32 bits de maneira específica, com campos como registradores, constantes imediatas, opcodes e códigos de função.

- **Formato R:** Usado para operações aritméticas e lógicas entre registradores. Contém campos para dois registradores de origem (`rs1` e `rs2`), um registrador de destino (`rd`), além de `funct3`, `funct7` e `opcode`.
- **Formato I:** Utilizado para instruções com operandos imediatos, como cargas de memória, operações aritméticas com constantes e saltos indiretos. Possui campos para `rd`, `rs1`, `funct3`, `imm` (12 bits) e `opcode`.
- **Formato S:** Empregado nas instruções de armazenamento (store), onde o valor de um registrador é escrito na memória. O campo imediato é dividido entre duas posições na instrução e, junto com `rs1`, `rs2` e `funct3`, define o endereço e o valor a ser armazenado.

- **Formato B:** Define as instruções de desvio condicional, como BEQ e BNE. O campo imediato, também distribuído em várias posições, determina o deslocamento relativo ao PC. Contém *rs1*, *rs2*, *funct3*, *imm* e *opcode*.
- **Formato U:** Utilizado por instruções que carregam constantes de 20 bits em registradores, como LUI e AUIPC. Os bits superiores da instrução representam o valor imediato, enquanto os campos *rd* e *opcode* completam a codificação.
- **Formato J:** Projetado para instruções de salto incondicional como JAL. O campo imediato representa um deslocamento de 20 bits relativo ao PC, codificado de forma não contígua.

Cada formato atende a diferentes necessidades do conjunto de instruções, garantindo versatilidade com simplicidade de decodificação. As tabelas a seguir apresentam os campos e o posicionamento de bits para cada tipo.

3.3.3 Modos de Endereçamento

O RISC-V utiliza modos de endereçamento simples para manter a eficiência e a uniformidade do conjunto de instruções:

3.3.3.1 Registrador

Todas as operações aritméticas e lógicas utilizam operandos vindos de registradores, cujo índice é codificado nos campos *rs1* e *rs2*.

3.3.3.2 Imediato

Valores constantes de até 12 bits são codificados diretamente na instrução do tipo I, facilitando instruções como ADDI e ORI.

3.3.3.3 Base-Deslocamento

Para acesso à memória, o endereço efetivo é calculado somando o conteúdo de um registrador *rs1* a um imediato de 12 bits (LW, SW).

3.3.3.4 Relativo ao PC

Saltos e desvios usam endereços relativos ao contador de programa (PC), codificados em campos de imediato (tipos B e J), permitindo saltos de curto e médio alcance.

3.3.3.5 Direto

No modo direto, utilizado principalmente em instruções de salto do tipo J, o campo imediato codificado na instrução fornece parte do endereço de destino de forma literal. A operação concatena os bits de imediato (20 bits) com os 12 bits mais significativos do contador de programa atual (PC), formando o endereço completo de 32 bits. Esse mecanismo permite saltos para posições conhecidas em tempo de projeto, sem cálculo adicional em registradores.

3.4 Linguagem de Descrição de Hardware

A Linguagem de Descrição de Hardware (HDL) é a ferramenta principal para modelar e validar circuitos digitais antes da síntese física. Em HDL, descreve-se o comportamento funcional (módulo comportamental) ou a interconexão estrutural (módulo estrutural) de componentes como registradores, ALU e unidades de controle. As linguagens mais utilizadas incluem:

- **Verilog:** sintaxe inspirada em C, popular em projetos industriais de FPGA e ASIC.
- **VHDL:** forte tipagem e verbosidade, ideal para verificação formal e documentação extensiva.

A simulação em HDL permite detectar e corrigir comportamentos incorretos antes de gerar o bitstream para FPGA.

3.5 *Field Programmable Gate Array* (FPGA)

O FPGA é um circuito integrado programável que contém blocos lógicos configuráveis (CLBs), memória embutida e matriz de interconexões programáveis. A síntese de um projeto HDL gera um arquivo de bitstream que define:

- **LUTs (Look-Up Tables):** implementam funções lógicas customizadas.
- **Flip-flops e registradores:** armazenam sinais e estados.
- **Rede de roteamento:** interconecta logicamente os CLBs conforme o design.
- **Blocos de I/O e BRAM:** facilitam comunicação externa e armazenamento temporário.

A flexibilidade de reconfiguração faz do FPGA um ambiente ideal para prototipação de processadores RISC-V monociclo, combinando alta frequência de clock e validação rápida de design.

4 Desenvolvimento

4.1 Características do Processador

O processador RISC-V ([PATTERSON; WATERMAN, 2019](#)) projetado neste trabalho será implementado no modelo monociclo e terá como base as seguintes características:

- Todas as instruções possuem 32 bits de comprimento;
- São utilizados 3 bits para o campo `opcode`, 4 bits para o campo `funct`, e os demais 25 bits são distribuídos entre registradores e imediatos;
- Serão implementados 64 registradores de uso geral, com largura de 32 bits;
- A arquitetura de memória segue o modelo Harvard, com memórias separadas para instruções e dados;
- Os tipos de instruções implementados serão: R, I, J e B;
- Cada tipo de instrução segue um formato fixo, com layout de campos definido.

Essas definições visam garantir simplicidade de decodificação, clareza na implementação do caminho de dados e compatibilidade com o subconjunto base da arquitetura RISC-V.

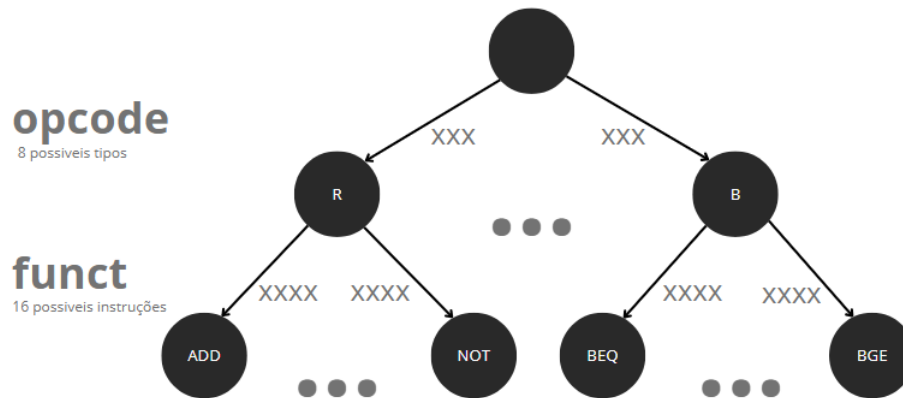
4.2 Organização do `opcode` e `funct`

A identificação das instruções no processador RISC-V deste projeto é feita de forma hierárquica, a partir da decodificação dos campos `opcode` e `funct`, presentes em todas as instruções de 32 bits.

O campo `opcode`, com 3 bits, é responsável por determinar o tipo da instrução (R, I, J ou B). Com base nessa definição, o campo `funct`, com 4 bits, é então utilizado para identificar qual operação específica deve ser executada dentro daquele tipo.

Essa organização facilita a implementação da unidade de controle, pois permite separar a lógica de decodificação por tipo de instrução e, dentro de cada tipo, atribuir uma operação específica a cada valor de `funct`.

Figura 2 – Árvore de decisão para decodificação baseada em opcode e funct



Fonte: O Autor

A [Figura 2](#) mostra a árvore de decisão usada para essa decodificação. Cada ramo do primeiro nível representa um valor possível do campo **opcode** (totalizando até 8 possibilidades). Cada ramo subsequente, vinculado a um valor de **opcode**, indica as operações disponíveis conforme os 16 possíveis valores de **funct**. Esse modelo torna a estrutura lógica do processador mais modular e facilita futuras expansões do conjunto de instruções.

4.3 Conjunto de Instruções

O conjunto de instruções adotado neste projeto é baseado no subconjunto RV32I da arquitetura RISC-V, sendo suficiente para executar operações aritméticas, lógicas, de desvio condicional, salto incondicional e acesso à memória. As instruções são classificadas em quatro formatos principais, de acordo com a estrutura de seus campos: Tipo R, Tipo I, Tipo J e Tipo B.

Cada formato apresenta um layout fixo de 32 bits, com campos específicos para registradores, imediatos e códigos de operação. Essa padronização facilita a decodificação e o controle interno do processador monociclo.

4.3.1 Tipo R

Utilizado para instruções aritméticas e lógicas que operam exclusivamente sobre registradores. Seu formato está descrito na [Tabela 1](#).

Tabela 1 – Formato das Instruções do Tipo R

Tamanho (bits)	7	6	6	6	4	3
Campo	null	rs2	rs1	rd	funct	opcode

Fonte: O Autor

As instruções do tipo R são utilizadas para operações aritméticas e lógicas entre registradores. Cada uma dessas instruções é identificada por um valor específico no campo **funct**, enquanto o campo **opcode** permanece fixo. A seguir, apresenta-se a codificação utilizada neste projeto para essas instruções:

Tabela 2 – Instruções do Tipo R implementadas

Instrução	funct (4 bits)	opcode (3 bits)
ADD	0000	000
SUB	0001	000
MULT	0010	000
DIV	0011	000
AND	0100	000
OR	0101	000
XOR	0110	000
NOT	0111	000

Fonte: O Autor

4.3.2 Tipo I

Empregado em instruções que utilizam valores imediatos, como operações aritméticas com constante, carregamento da memória e leitura de entrada. O formato pode ser consultado na [Tabela 3](#).

Tabela 3 – Formato das Instruções do Tipo I

Tamanho (bits)	13	6	6	4	3
Campo	im	rs1	rd	funct	opcode

Fonte: O Autor

As instruções do tipo I são utilizadas para operações que envolvem valores imediatos, acessos à memória e manipulação de registradores. Elas utilizam um campo de 13 bits para o valor imediato, dois registradores (um de origem e um de destino), um campo de função com 4 bits e um **opcode** de 3 bits.

A tabela abaixo apresenta as instruções do tipo I implementadas neste projeto:

Tabela 4 – Instruções do Tipo I implementadas

Instrução	funct (4 bits)	opcode (3 bits)
LOAD	0000	001
STORE	0001	001
ADDI	0010	001
SUBI	0011	001
ANDI	0100	001
ORI	0101	001
XORI	0110	001
LOAD ADDRESS	0111	001
MOV	1000	001

Fonte: O Autor

4.3.2.1 Tipo J

As instruções do tipo J são utilizadas para realizar saltos incondicionais no fluxo de execução do programa. O campo imediato de 19 bits representa o deslocamento relativo ao valor atual do PC (Program Counter). Essas instruções também utilizam um registrador de destino para armazenar o endereço de retorno, quando necessário, além dos campos **funct** e **opcode**.

Tabela 5 – Formato das Instruções do Tipo J

Tamanho (bits)	19	6	4	3
Campo	imm	rd	funct	opcode

Fonte: O Autor

A seguir, as instruções do tipo J utilizadas neste projeto:

Tabela 6 – Instruções do Tipo J implementadas

Instrução	funct (4 bits)	opcode (3 bits)
JUMP	0000	010
JR	0001	010
JAL	0010	010

Fonte: O Autor

4.3.2.2 Tipo B

As instruções do tipo B são responsáveis por desvios condicionais. A decisão de desviar ou não para outro endereço é baseada na comparação entre dois registradores. O campo imediato de 13 bits define o deslocamento relativo ao contador de programa (PC), enquanto os campos **rs1** e **rd** contêm os registradores a serem comparados. A combinação de **funct** e **opcode** define a condição de comparação.

Tabela 7 – Formato das Instruções do Tipo B

Tamanho (bits)	13	6	6	4	3
Campo	im	rs1	rd	funct	opcode

Fonte: O Autor

A tabela abaixo mostra as instruções do tipo B utilizadas neste projeto:

Tabela 8 – Instruções do Tipo B implementadas

Instrução	funct (4 bits)	opcode (3 bits)
BEQ	0000	011
BNE	0001	011
BLT	0010	011
BLE	0011	011
BGT	0100	011
BGE	0101	011

Fonte: O Autor

4.3.2.3 Tipo N e Tipo IO

Para lidar com o controle do processador e a comunicação com periféricos externos, foram criados dois formatos de instrução adicionais: Tipo N (Nenhuma Operação) e Tipo IO (Entrada/Saída).

Tipo N

As instruções do Tipo N são utilizadas para operações especiais que não se enquadram nos formatos de dados convencionais, como o controle do estado do processador.

Tabela 9 – Formato das Instruções do Tipo N

Tamanho (bits)	25	4	3
Campo	null	funct	opcode

Fonte: O Autor

As duas instruções do Tipo N implementadas são para não realizar operação (‘NOP’) e para parar o processador (‘HALT’).

Tabela 10 – Instruções do Tipo N implementadas

Instrução	funct (4 bits)	opcode (3 bits)
NOP	0000	100
HALT	0001	100

Fonte: O Autor

Tipo IO

As instruções do Tipo IO foram definidas para gerenciar a Entrada e Saída de dados entre o processador e o mundo externo, como switches e displays. O campo de 6 bits central é utilizado como registrador de destino (**rd**) para a instrução ‘IN’ e como registrador de origem (**rs1**) para a instrução ‘OUT’.

Tabela 11 – Formato das Instruções do Tipo IO

Tamanho (bits)	19	6	4	3
Campo	null	rd / rs1	funct	opcode

Fonte: O Autor

As instruções implementadas permitem ler dados para um registrador e escrever o conteúdo de um registrador para a saída.

Tabela 12 – Instruções do Tipo IO implementadas

Instrução	funct (4 bits)	opcode (3 bits)
IN	0000	110
OUT	0001	110

Fonte: O Autor

As instruções foram escolhidas de modo a atender às operações básicas necessárias para execução de programas simples e validação do funcionamento do processador monociclo em FPGA.

Tabela 13 – Resumo das Instruções Implementadas

Instrução	Operação	Descrição
Tipo R		
ADD	$rd = rs1 + rs2$	Adiciona rs1 e rs2
SUB	$rd = rs1 - rs2$	Subtrai rs2 de rs1
MULT	$rd = rs1 * rs2$	Multiplica rs1 por rs2
DIV	$rd = rs1 / rs2$	Divide rs1 por rs2 (inteiro)
AND	$rd = rs1 \& rs2$	AND bit a bit entre rs1 e rs2
OR	$rd = rs1 rs2$	OR bit a bit entre rs1 e rs2
XOR	$rd = rs1 \wedge rs2$	XOR bit a bit entre rs1 e rs2
NOT	$rd = \sim rs1$	NOT bit a bit de rs1
Tipo I		
LOAD	$rd = \text{MEM}[rs1 + \text{imm}]$	Carrega dado da memória para rd
STORE	$\text{MEM}[rs1 + \text{imm}] = rs2$	Armazena dado de rs2 na memória
ADDI	$rd = rs1 + \text{imm}$	Adiciona imediato a rs1
SUBI	$rd = rs1 - \text{imm}$	Subtrai imediato de rs1
ANDI	$rd = rs1 \& \text{imm}$	AND bit a bit entre rs1 e imediato
ORI	$rd = rs1 \text{imm}$	OR bit a bit entre rs1 e imediato
XORI	$rd = rs1 \wedge \text{imm}$	XOR bit a bit entre rs1 e imediato
LOAD ADDRESS	$rd = \text{endereço}(\text{label})$	Carrega endereço de um rótulo no registrador
MOV	$rd = rs1$	Move valor de rs1 para rd
Tipo J		
JUMP	$PC = \text{label}$	Salto incondicional para um rótulo
JR	$PC = rs1$	Salto incondicional para endereço em registrador
JAL	$rd = PC + 1; PC = PC + \text{imm}$	Salta para offset e salva endereço de retorno
Tipo B		
BEQ	$PC = (rs1 == rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 igual a rs2
BNE	$PC = (rs1 != rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 diferente de rs2
BLT	$PC = (rs1 < rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 menor que rs2
BLE	$PC = (rs1 \leq rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 menor ou igual a rs2
BGT	$PC = (rs1 > rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 maior que rs2
BGE	$PC = (rs1 \geq rs2) ? PC + \text{imm} : PC + 1$	Desvia se rs1 maior ou igual a rs2

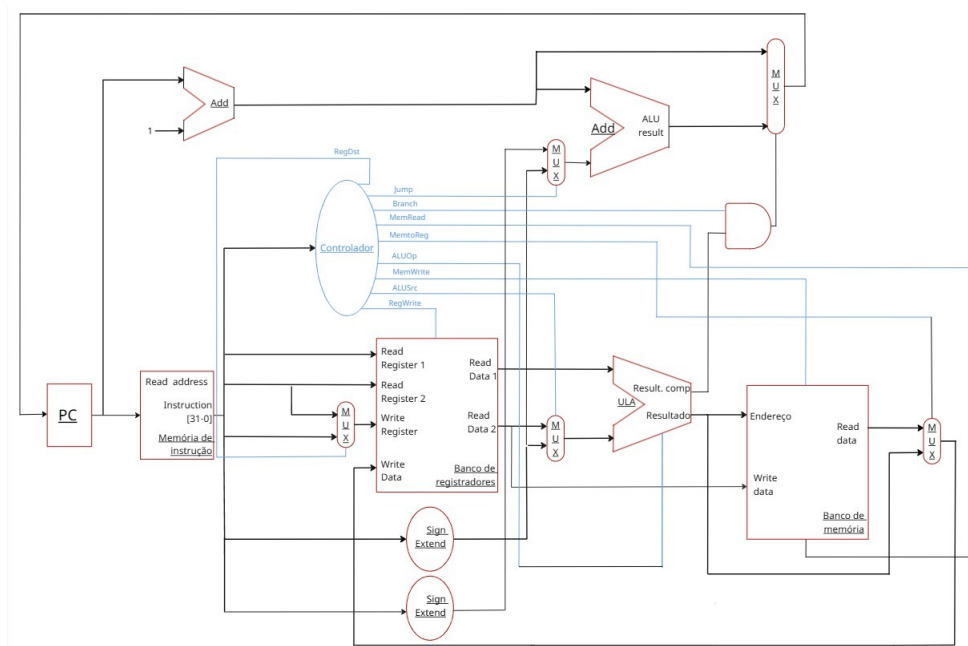
Fonte: O Autor

4.4 Caminho de Dados

O caminho de dados do processador define como os dados se movem entre os componentes internos — como registradores, ALU, memória e unidade de controle — durante a execução das instruções. A seguir, são apresentados os diagramas que representam o caminho de dados geral e os caminhos específicos utilizados para a execução de cada tipo de instrução.

4.4.1 Caminho de Dados Geral

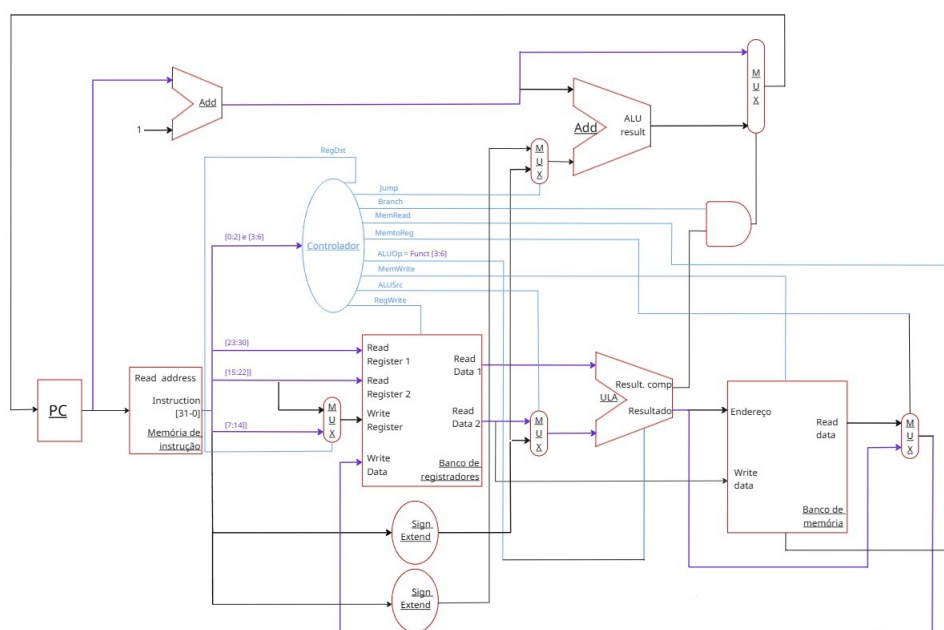
Figura 3 – Caminho de dados geral do processador



Fonte: O Autor

4.4.2 Caminho de Dados para Instruções do Tipo R

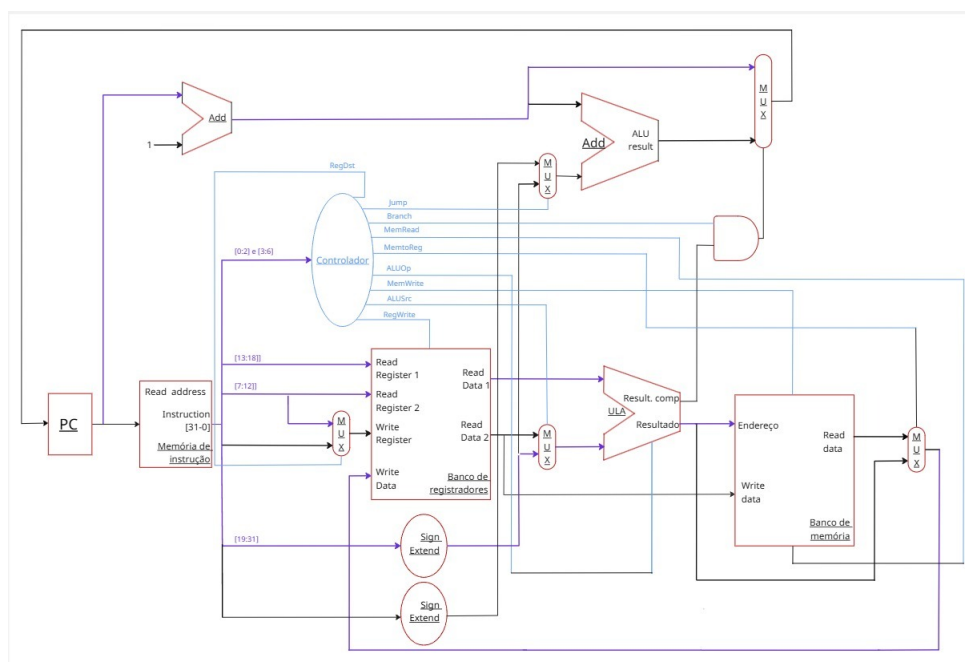
Figura 4 – Caminho de dados para instruções do tipo R



Fonte: O Autor

4.4.3 Caminho de Dados para Instruções do Tipo I

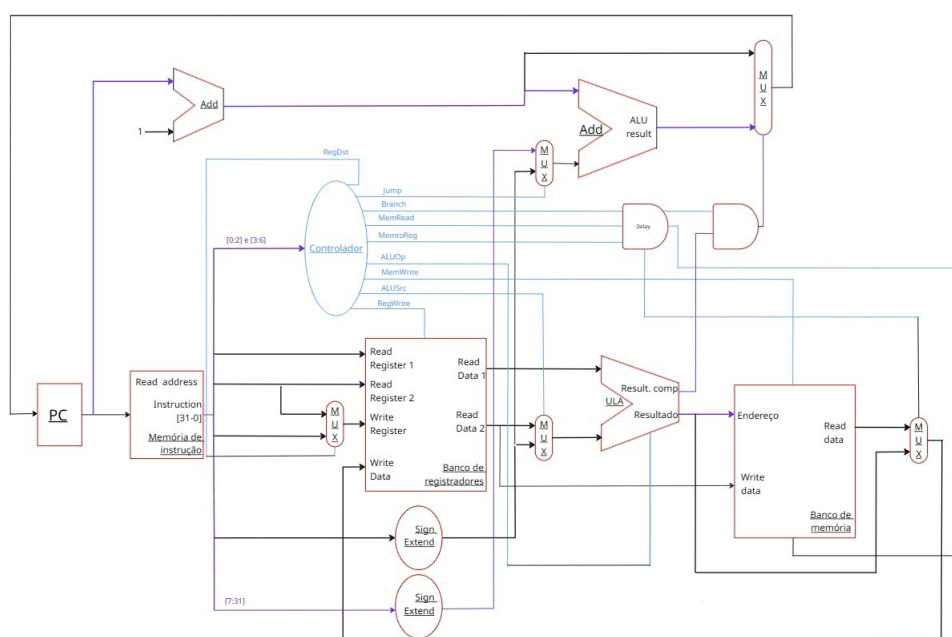
Figura 5 – Caminho de dados para instruções do tipo I



Fonte: O Autor

4.4.4 Caminho de Dados para Instruções do Tipo J

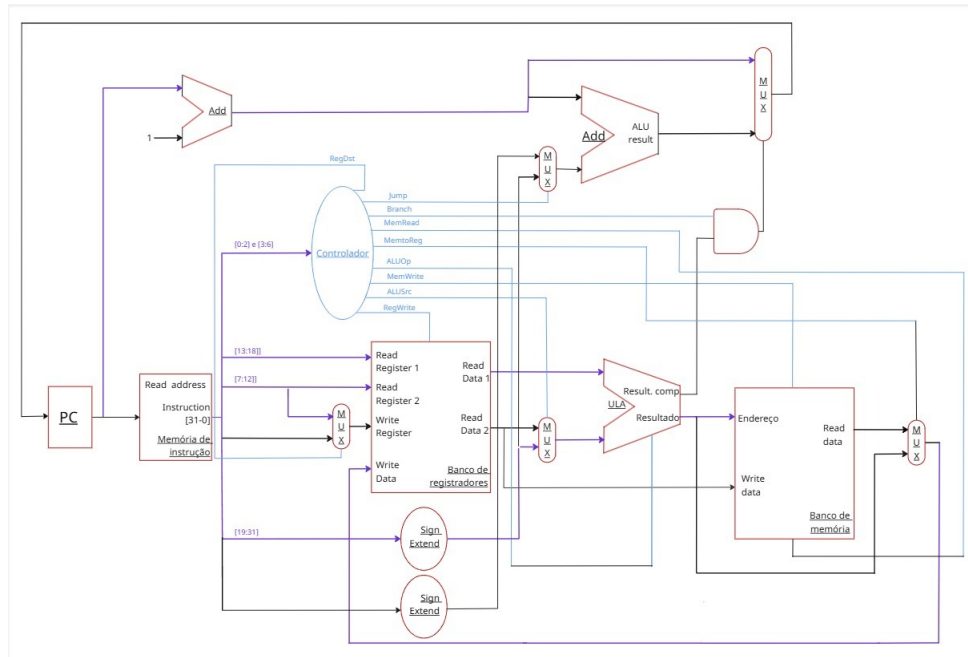
Figura 6 – Caminho de dados para instruções do tipo J



Fonte: O Autor

4.4.5 Caminho de Dados para Instruções do Tipo B

Figura 7 – Caminho de dados para instruções do tipo B



Fonte: O Autor

4.5 Implementação

Esta seção descreve os códigos Verilog desenvolvidos para a implementação do processador RISC-V monociclo em FPGA. Serão detalhados os módulos principais, começando pela Unidade Lógica Aritmética (ULA), e progressivamente os demais componentes que formam o caminho de dados e a unidade de controle do processador.

4.5.1 Unidade Lógica Aritmética (ULA)

A Unidade Lógica Aritmética (ULA) é responsável por executar as operações aritméticas e lógicas, além das comparações para as instruções de desvio condicional. A ULA implementada neste projeto é capaz de realizar adição, subtração, multiplicação, divisão inteira, operações AND, OR, XOR e NOT, bem como as comparações para os branches BEQ, BNE, BLT, BLE, BGT e BGE. O código Verilog para a ULA é apresentado a seguir:

```

1  module ula_32bits (
2      input  signed [31:0] rs1,          // Operando 1
3      input  signed [31:0] rs2,          // Operando 2
4      input   [3:0]      op,             // Código da operação
5      input              branch_en,      // Sinal de controle de branch
6      output reg signed [31:0] rd,        // Resultado da operação aritmética/lógica
7      output reg          branch_taken   // Resultado da comparação de branch
8  );
9
10
11
12  always @(*) begin
13
14      case (op)
15          4'b0000: rd = rs1 + rs2;        // ADD
16          4'b0001: rd = rs1 - rs2;        // SUB
17          4'b0010: rd = rs1 * rs2;        // MULT
18          4'b0011: rd = rs1 / rs2;        // DIV
19          4'b0100: rd = rs1 & rs2;        // AND
20          4'b0101: rd = rs1 | rs2;        // OR
21          4'b0110: rd = rs1 ^ rs2;        // XOR
22          4'b0111: rd = ~rs1;             // NOT
23          default: rd = 32'b0;            // Padrão
24      endcase
25
26
27      if (branch_en) begin
28          case (op)
29              4'b0000: branch_taken = (rs1 == rs2);        // BEQ
30              4'b0001: branch_taken = (rs1 != rs2);        // BNE
31              4'b0010: branch_taken = (rs1 < rs2);        // BLT
32              4'b0011: branch_taken = (rs1 <= rs2);       // BLE
33              4'b0100: branch_taken = (rs1 > rs2);        // BGT
34              4'b0101: branch_taken = (rs1 >= rs2);       // BGE
35              default: branch_taken = 1'b0;
36          endcase
37      end else begin
38          branch_taken = 1'b0; // Branch não habilitado
39      end
40  end
41
42  endmodule

```

Listing 4.1 – Código Verilog da ULA

4.5.2 Banco de Registradores

O Banco de Registradores é um componente crucial para o armazenamento temporário de dados utilizados pelo processador. Este módulo implementa 64 registradores de uso geral, cada um com 32 bits de largura. O registrador $x0$ (endereço 6'b0) é hardwired para sempre retornar zero, e tentativas de escrita nesse registrador são ignoradas, seguindo a especificação RISC-V (PATTERSON; WATERMAN, 2019). A escrita nos registradores é síncrona com a borda de subida do clock, enquanto a leitura é assíncrona, proporcionando acesso rápido aos dados.

```
1 module banco_registradores (
2     input      clk,          // Clock
3     input      we,          // Write Enable (habilita escrita)
4     input [5:0] rs1_addr,    // Endereço do registrador fonte 1
5     input [5:0] rs2_addr,    // Endereço do registrador fonte 2
6     input [5:0] rd_addr,     // Endereço do registrador destino
7     input [31:0] rd_data,     // Dados a serem escritos
8     output [31:0] rs1_data,   // Dados lidos do registrador fonte 1
9     output [31:0] rs2_data    // Dados lidos do registrador fonte 2
10 );
11
12 // Array de 64 registradores de 32 bits cada
13 reg [31:0] registradores [0:63];
14
15 // Escrita síncrona
16 always @(posedge clk) begin
17     if (we) begin
18         if (rd_addr != 6'b0) begin // r0 sempre é 0
19             registradores[rd_addr] <= rd_data;
20         end
21     end
22 end
23
24
25 // Registrador r0 sempre retorna 0
26 assign rs1_data = (rs1_addr == 6'b0) ? 32'b0 : registradores[rs1_addr];
27 assign rs2_data = (rs2_addr == 6'b0) ? 32'b0 : registradores[rs2_addr];
28
29 endmodule
```

Listing 4.2 – Código Verilog do Banco de Registradores

4.5.3 Memória RAM

A memória RAM (Random Access Memory) é utilizada para armazenar os dados do programa, seguindo o modelo de memória Harvard (PATTERSON; HENNESY, 2006), onde instruções e dados possuem espaços de memória separados. Este módulo implementa uma memória de 64 posições, com cada posição armazenando 32 bits de dados. A leitura é feita de forma assíncrona, mas a escrita é feita de forma síncrona, garantindo a sincronização com os demais componentes do processador monociclo.

```
1 module memoria_ram
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=6)
3   (
4       input [(DATA_WIDTH-1):0] data,
5       input [(ADDR_WIDTH-1):0] read_addr, write_addr,
6       input we, clk,
7       output reg [(DATA_WIDTH-1):0] q
8   );
9
10
11   reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];
12
13   always @ (posedge clk)
14   begin
15
16       if (we)
17           ram[write_addr] <= data;
18
19   end
20
21   always @ (*)
22   begin
23
24       q <= ram[read_addr];
25
26   end
27 endmodule
```

Listing 4.3 – Código Verilog da Memória RAM

4.5.4 Memória de Programa

A Memória de Programa é um componente vital na arquitetura de Harvard do processador (PATTERSON, 2017), sendo responsável por armazenar as instruções a serem executadas. Este módulo é configurado para ter 64 posições de memória, onde cada posição armazena uma instrução de 32 bits. O endereço é fornecido pelo Program Counter (PC), e o módulo carrega as instruções de um arquivo txt ('programa.txt') durante a inicialização.

```

1 module memoria_programa
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=6)
3   (
4       input [(ADDR_WIDTH-1):0] addr,
5       input clk,
6       output reg [(DATA_WIDTH-1):0] q
7   );
8
9       reg [DATA_WIDTH-1:0] rom[2**ADDR_WIDTH-1:0];
10
11
12       initial
13       begin
14           $readmemb("programa.txt", rom);
15       end
16
17       always @ (posedge clk)
18       begin
19           q <= rom[addr];
20       end
21
22 endmodule

```

Listing 4.4 – Código Verilog da Memória de Programa

4.5.5 Módulos de Interface e Controle

Além dos componentes centrais do caminho de dados, foram desenvolvidos módulos específicos para gerenciar a interface com o hardware externo (a placa FPGA) e para garantir a estabilidade do sistema.

4.5.5.1 Sistema de Entrada e Saída (I/O)

Para permitir a interação do usuário com o processador, foi implementado um sistema de Entrada e Saída (I/O) customizado, controlado pelas novas instruções IN e OUT.

A lógica de entrada é baseada em um mecanismo de *stall* (pausa). Ao decodificar uma instrução IN, a unidade de controle ativa um sinal que congela o Program Counter (PC). O processador então aguarda um sinal de um botão físico externo, que é sincronizado pelo módulo `ControleSinal`. Apenas quando o botão é pressionado, o valor presente nos switches da placa é capturado, a pausa é liberada e a execução continua.

A lógica de saída foi projetada para fornecer uma visualização persistente dos resultados. O módulo `ControleSaida` recebe o dado de 32 bits da instrução `OUT` e o armazena em um registrador interno (latch). Este módulo é responsável por converter o valor binário armazenado para dígitos decimais e acionar os 8 displays de 7 segmentos. Graças a esse registrador, o último valor enviado permanece visível mesmo após o término ou parada do programa.

4.5.5.2 Gerenciamento de Clock

O processador foi projetado como uma arquitetura monociclo, na qual cada instrução deve ser concluída em um único ciclo de clock[cite: 131]. O tempo de ciclo, portanto, é limitado pelo caminho crítico do processador — a sequência de operações combinacionais mais longa, tipicamente encontrada na instrução `LOAD`.

As placas FPGA possuem osciladores de alta frequência, como 50 MHz[cite: 283], cujo período (20 ns) é insuficiente para que os sinais se propaguem por todo o caminho de dados de forma estável. Para garantir o funcionamento correto, foi essencial gerar um clock de frequência mais baixa. A solução adotada foi o uso de um módulo divisor de frequência, implementado através de um PLL (*Phase-Locked Loop*) na ferramenta de síntese, para gerar um clock estável de 1 MHz a partir da entrada de 50 MHz, garantindo a integridade temporal de todas as operações.

4.5.5.3 Inicialização do Processador (Reset)

Garantir que o processador inicie sua execução em um estado conhecido — especificamente, com o Program Counter (PC) no endereço 0 — é fundamental. Em vez de utilizar um pino de reset externo, que exigiria uma interação manual a cada inicialização, optou-se por um sistema de **auto-inicialização** para o PC.

Esta solução foi implementada diretamente no módulo do PC, onde o registrador interno que armazena o endereço é declarado com um valor inicial: `reg [31:0] PC_reg = 32'h0;`. Este é um recurso sintetizável que instrui a ferramenta da FPGA a usar os circuitos de reset globais da placa, que são ativados quando ela é energizada, para carregar o valor zero no `PC_reg`. Essa abordagem simplifica a interface do processador com o mundo externo, tornando-o mais robusto e autônomo, sem a necessidade de um botão de reset físico para o funcionamento correto.

4.6 Integração dos Módulos

A etapa final da implementação do processador RISC-V ([PATTERSON; WATTERMANN, 2019](#)) monociclo consiste na integração de todos os módulos desenvolvidos — a Unidade Lógica Aritmética (ULA), o Banco de Registradores, a Memória RAM e a Memória de Programa — para compor a arquitetura completa do processador, capaz de decodificar e executar instruções em um único ciclo de clock.

Essa fase inclui a interligação lógica das entradas e saídas de cada módulo, formando o caminho de dados principal do processador. O Program Counter (PC) fornece o endereço para busca na Memória de Programa, que retorna a instrução de 32 bits a ser processada. Em seguida, a instrução é encaminhada para o estágio de decodificação.

Durante essa decodificação, os campos da instrução — como os registradores de origem (rs1 e rs2) e o de destino (rd) — são usados para acessar o Banco de Registradores. Os dados lidos são então enviados para a ULA, juntamente com eventuais valores imediatos extraídos da instrução. Com isso, foi implementada a integração direta entre a ULA e o Banco de Registradores, permitindo que os operandos sejam processados imediatamente para a execução da operação aritmética ou lógica necessária.

O resultado produzido pela ULA pode ter destinos diferentes, dependendo da instrução: pode ser escrito de volta no Banco de Registradores, como nas instruções aritméticas; pode ser usado como endereço efetivo para acesso à Memória RAM, como nas instruções LOAD e STORE; ou pode ser usado para cálculo do próximo valor do PC em instruções de controle de fluxo, como saltos e desvios condicionais.

Todos os módulos operam de forma coordenada por meio de sinais de controle que determinam o comportamento de cada unidade, assegurando que a instrução seja executada corretamente dentro de um único ciclo de clock. A integração da ULA com o Banco de Registradores foi um passo fundamental para viabilizar esse fluxo contínuo de dados e garantir o funcionamento correto do processador monociclo.

```
1 module integração (
2     input      clk,          // Clock
3     input      we,          // Write Enable (habilita escrita)
4     input [5:0] rs1_addr,    // Endereço do registrador fonte 1
5     input [5:0] rs2_addr,    // Endereço do registrador fonte 2
6     input [5:0] rd_addr,     // Endereço do registrador destino
7     input [3:0] op,          // Operação da ULA
8     input      branch_en,    // Sinal de controle de branch
9     input [31:0] rd_data,     // Dados a serem escritos no banco (pode vir da ULA ou de
                             fora)
10    output [31:0] rs1_data,    // Dados lidos do registrador fonte 1
11    output [31:0] rs2_data,    // Dados lidos do registrador fonte 2
12    output [31:0] ula_result,  // Saída da ULA
13    output      branch_taken  // Resultado da comparação
14 );
15
16
17 wire signed [31:0] op1;
18 wire signed [31:0] op2;
19
20 assign op1 = rs1_data;
21 assign op2 = rs2_data;
22
23
24 banco banco(
25     .clk(clk),
26     .we(we),
27     .rs1_addr(rs1_addr),
28     .rs2_addr(rs2_addr),
29     .rd_addr(rd_addr),
30     .rd_data(rd_data),
31     .rs1_data(rs1_data),
32     .rs2_data(rs2_data)
33 );
34
35 // Inst ncia da ULA
36 ula ula (
37     .rs1(op1),
38     .rs2(op2),
39     .op(op),
40     .branch_en(branch_en),
41     .rd(ula_result),
42     .branch_taken(branch_taken)
43 );
44
45 endmodule
```

Listing 4.5 – Código Verilog da Integração da ULA com o Banco de Registradores

5 Resultados Obtidos e Discussões

Inicialmente, o foco deste trabalho esteve no planejamento detalhado da arquitetura do processador RISC-V ([PATTERSON; WATERMAN, 2019](#)) e na definição precisa do conjunto de instruções a serem suportadas. Após a conclusão das implementações dos módulos individuais em Verilog, o próximo passo consistiu na realização de testes funcionais extensivos. Esses testes foram conduzidos por meio de simulações, onde foram geradas *waveforms* (formas de onda) para verificar o comportamento esperado de cada componente e a correta execução das operações.

5.0.1 Waveform da Unidade Lógica Aritmética (ULA)

Esta subseção apresenta as formas de onda obtidas a partir da simulação do módulo *ula_32bits.v*, demonstrando a funcionalidade das operações aritméticas, lógicas e de comparação implementadas. A Figura 8 ilustra exemplos de entradas e saídas que validam o correto funcionamento da ULA para diferentes códigos de operação (*op*) e estados do sinal *branch_en*.

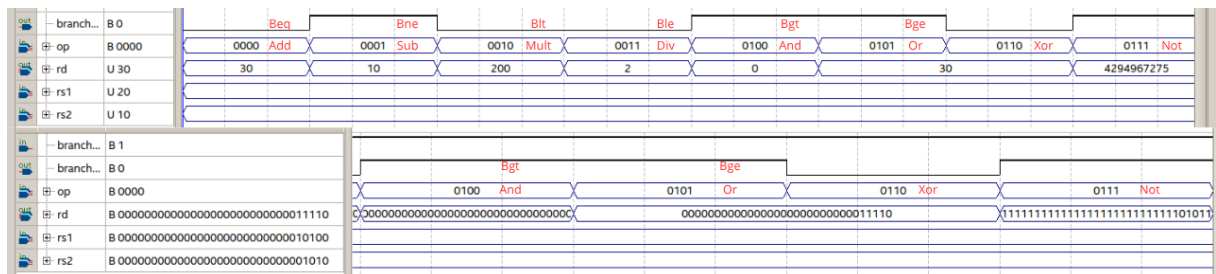


Figura 8 – Waveform de Simulação da ULA.

5.0.2 Waveform do Banco de Registradores

A simulação do módulo *banco_registradores.v* permitiu verificar a correta leitura e escrita dos dados nos 64 registradores de 32 bits, além de confirmar que o registrador *x0* permanece sempre com o valor zero, mesmo em tentativas de escrita. A Figura 9 exibe a waveform que comprova a operação síncrona de escrita e a leitura assíncrona dos dados.

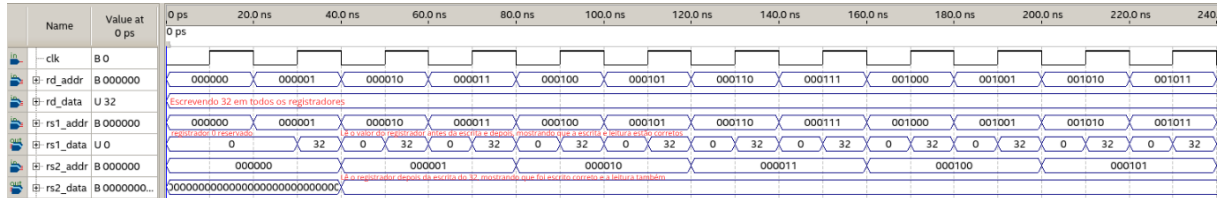


Figura 9 – Waveform de Simulação do Banco de Registradores.

5.0.3 Waveform da Memória RAM

As formas de onda geradas para o módulo *memoria_ram.v* validam o comportamento síncrono de escrita e leitura da memória de dados. A Figura 10 mostra a interação entre os sinais de clock, escrita (*we*), endereço (*addr*), dados de entrada (*data_in*) e dados de saída (*data_out*), confirmando que a memória armazena e recupera os dados corretamente.

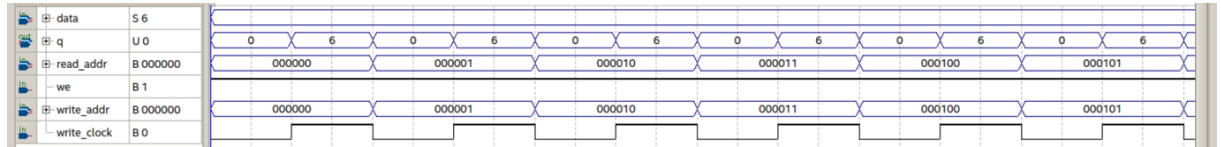


Figura 10 – Waveform de Simulação da Memória RAM.

5.0.4 Waveform da Memória de Programa

A simulação do módulo *memoria_programa.v* teve como objetivo verificar a correta leitura das instruções a partir do endereço fornecido pelo PC, além de confirmar o carregamento do arquivo hexadecimal na inicialização. A Figura 11 ilustra a relação entre o endereço do PC e a instrução de 32 bits fornecida pelo módulo.

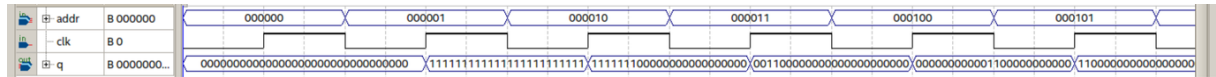


Figura 11 – Waveform de Simulação da Memória de Programa.

5.0.5 Waveform da Integração ULA com Banco de Registradores

Para validar a comunicação entre componentes essenciais, foi realizada uma simulação da integração funcional do Banco de Registradores com a ULA. Esta waveform, apresentada na Figura 12, demonstra o fluxo de dados dos operandos lidos do Banco de Registradores para a ULA, e o resultado da operação da ULA sendo escrito de volta no Banco de Registradores. Este teste é fundamental para garantir que as operações de registro-registro, por exemplo, ocorram conforme o esperado no caminho de dados.

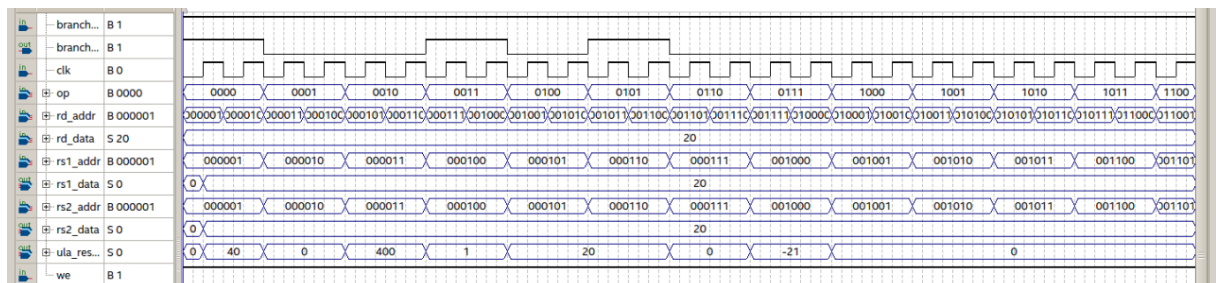


Figura 12 – Waveform de Simulação da Integração da ULA com o Banco de Registradores.

5.0.6 Validação Funcional do Processador Integrado em FPGA

Após a verificação individual dos componentes por meio de simulação, a etapa final e mais crítica consistiu na validação do processador totalmente integrado no kit FPGA. Este teste prático teve como objetivo não apenas confirmar a lógica de cada módulo, mas também a correta interação entre o caminho de dados, a Unidade de Controle e o sistema de Entrada e Saída (I/O) desenvolvido.

Para esta validação abrangente, foram criados dois programas de teste completos, projetados para utilizar uma vasta gama do conjunto de instruções implementado. O primeiro programa calcula a **sequência de Fibonacci**. Ele inicia com a instrução IN,

que pausa o processador e exibe o valor dos switches em tempo real nos displays. Ao pressionar o botão de entrada, o número de iterações desejado (N) é capturado, e o processador executa um laço que utiliza instruções aritméticas (**ADD**), de desvio (**BGE**) e de salto (**JL**) para calcular cada termo da sequência. Cada resultado é exibido nos displays de 7 segmentos por meio da instrução **OUT**, demonstrando o funcionamento correto do sistema de saída.

O segundo programa implementado calcula a **potência** (a^b), testando funcionalidades adicionais. O sistema recebe duas entradas sequenciais com a instrução **IN**, separadas por um **NOP** para garantir a sincronização com a ação do usuário. O cálculo é realizado por um laço com a instrução de multiplicação (**MULT**). Ao final, para validar o caminho de dados da memória, o resultado é primeiro salvo em uma posição da RAM com a instrução **STORE** e, em seguida, lido de volta para um novo registrador com **LOAD**, antes de ser finalmente exibido com **OUT**.

A execução bem-sucedida de ambos os programas na placa FPGA serviu como a validação funcional definitiva do projeto. A capacidade do processador de receber entradas, processar laços condicionais, realizar operações aritméticas, acessar a memória de dados e exibir os resultados corretamente demonstrou que todos os módulos, incluindo o sistema de I/O customizado, foram integrados com sucesso e operam conforme a arquitetura especificada.

6 Considerações Finais

O desenvolvimento deste projeto culminou com a implementação bem-sucedida de um processador RISC-V monociclo funcional, atingindo o objetivo geral de traduzir o conhecimento teórico de arquitetura de computadores em um sistema de hardware validado em uma placa FPGA. O projeto percorreu todas as etapas essenciais do design de um processador, desde a especificação da arquitetura e do conjunto de instruções até a integração final e os testes funcionais.

A arquitetura final se consolidou em um modelo Harvard monociclo, com 64 registradores de uso geral e um conjunto de instruções customizado, mas inspirado na base RV32I. Foram definidos e implementados os formatos de instrução R, I, J e B, além de um novo formato IO para gerenciar a comunicação com periféricos. Decisões de projeto importantes foram tomadas para garantir a robustez do sistema, como a implementação de um Program Counter com auto-inicialização para garantir um estado de partida conhecido sem a necessidade de um reset externo, e a utilização de um PLL para gerar um clock de operação estável e compatível com o tempo de propagação do caminho crítico do processador.

Todos os módulos fundamentais, incluindo a Unidade Lógica Aritmética (ULA), o Banco de Registradores, as memórias e a Unidade de Controle, foram codificados em Verilog e integrados em um processador coeso. A validação do projeto transcendeu as simulações de componentes individuais, que foram apresentadas nos resultados parciais. A funcionalidade do processador completo foi comprovada através da execução de programas de teste, como o cálculo da sequência de Fibonacci e de potência, que exigiram a correta operação de todo o caminho de dados e da lógica de controle. A implementação bem-sucedida de um sistema de entrada e saída interativo — com a leitura de dados de switches físicos e a exibição de resultados em displays de 7 segmentos — serviu como a validação final e prática de todo o sistema.

Conclui-se, portanto, que os objetivos propostos foram alcançados. O processador não apenas executa corretamente instruções aritméticas, lógicas, de acesso à memória e de desvio, mas também interage com o ambiente externo, demonstrando a viabilidade e a flexibilidade da arquitetura RISC-V. Como trabalhos futuros, sugere-se a expansão do processador para uma arquitetura *pipeline*, visando um aumento significativo de desempenho, e a adição de novas instruções para suportar extensões padrão do RISC-V, como a de multiplicação e divisão (Extensão 'M').

Referências

EDITORIAIS DA TMLT. *10 diferenças entre Von Neumann e Harvard architecture*. 2021. Acesso em: 14 maio 2022. Disponível em: <<https://www.currentschoolnews.com/pt/noticias-de-educacao/diferenca-entre-a-arquitetura-de-von-neumann-e-harvard/>>. Citado na página 10.

PATTERSON, D. *Organização e projeto de computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: GEN LTC, 2017. Citado na página 27.

PATTERSON, D.; WATERMAN, A. *Guia Prático RISC-V: Atlas de uma Arquitetura Aberta*. 1.0.0. ed. Berkeley, California: Strawberry Canyon LLC, 2019. Tradução para o português por Luiz Gustavo Xavier, Nathan Formentin e Marcelo Pias. ISBN 978-0-9992491-1-6. Citado 5 vezes nas páginas 11, 14, 25, 29 e 31.

PATTERSON, D. A.; HENNESY, J. L. *Computer architecture: a quantitative approach*. 3. ed. [S.l.]: Morgan Kaufmann, 2006. Citado na página 26.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson, 2017. Citado 2 vezes nas páginas 9 e 10.